

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.10

# Exceptions & Modules

*Day 11 — failing well, and using code other people wrote*

**90 min**

Concepts + live code

**30 min**

Practice notebook

**2**

Topics in one session

Prepared & presented by **Srinivasa Sai Chava**



Boston University

# Where we are

## You have already met eight exception types

|                   |       |                                  |
|-------------------|-------|----------------------------------|
| ValueError        | Day 1 | <code>int("abc")</code>          |
| TypeError         | Day 1 | <code>"10" + 5</code>            |
| NameError         | Day 3 | a variable that is not there     |
| UnboundLocalError | Day 3 | <code>count = count + 1</code>   |
| IndexError        | Day 4 | <code>marks[99]</code>           |
| KeyError          | Day 5 | <code>d["missing"]</code>        |
| AttributeError    | Day 7 | a missing <code>self.name</code> |
| FileNotFoundError | Day 9 | a file that is not there         |

## Today you stop being their victim

So far an exception has meant your program stops and you go back to fix the code. That is right during development. But a program that reads a file the user chose, or takes input you cannot control, has to keep working when something goes wrong.

**Today: decide what should happen instead of crashing — and raise your own errors when something is genuinely wrong.**

## Five minutes on the assessment first

Before we start, we will go through the three questions the class found hardest. If you scored below the class average, the fix is almost never “study more” — it is redoing the practice notebooks for the two or three days those questions came from. Come and see me at the break and I will tell you which ones.

# How today's two hours are structured

1

## Reading a traceback

What Python is actually telling you

15 min

2

## try / except / else / finally

Catching the right thing, and only that

25 min

3

## Raising your own

raise, and custom exception classes

20 min

4

## Modules & packages

import, the standard library, pip

25 min

5

## Mini build

A robust CSV loader

5 min

6

## Practice notebook

You code, I walk around

30 min

*The single most useful habit today: catch the SPECIFIC exception you expected, never everything.*

# By the end of today you will be able to

1

## Read

Work out what a traceback is telling you, and where to look

2

## Handle

Catch the exceptions you expect and let the rest through

3

## Raise

Signal a problem yourself, with your own exception types

4

## Import

Use the standard library and organise code into modules

### The principle behind all of it

An exception you did not expect should stop the program loudly, so you find out about it. An exception you DID expect should be handled quietly, so the program carries on. Most bad error handling comes from confusing the two — usually by catching everything and hiding a real bug for months.

1

# Reading a Traceback

What Python is telling you · 15 minutes

- The shape of a traceback
- Read it bottom-up
- The exception hierarchy

# A traceback is a map, not a telling-off

```
Traceback (most recent call last):  
  File "analysis.py", line 12, in <module>  
    result = average(marks)  
  File "analysis.py", line 7, in average  
    return total / len(values)  
ZeroDivisionError: division by zero
```



## Start at the BOTTOM

the last line names the error and describes it



## ZeroDivisionError

the TYPE — this is what you would catch



## division by zero

the message — what actually went wrong



## line 7, in average

where it broke



## line 12, in <module>

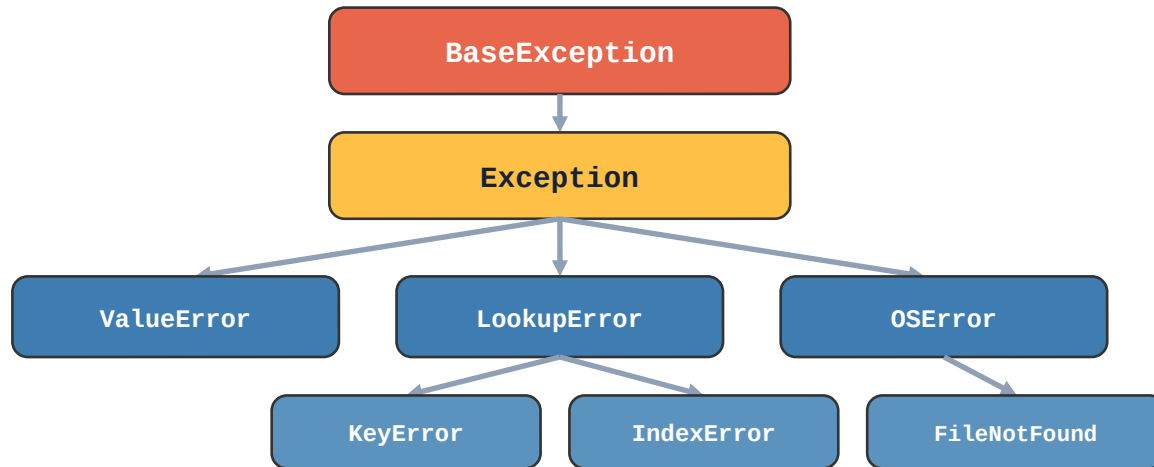
who called that — read upwards to trace it back

## “Most recent call last” is the instruction, not a description

The frames are printed oldest first, so the line that actually failed is at the bottom, just above the error. Read the last line, then work upwards only if you need to know who called it. Students who read tracebacks top-down waste a great deal of time on files they never wrote.

# Exceptions are classes — and they inherit

## A simplified family tree



## Why the tree matters

```
except LookupError:
    ...

# catches KeyError AND IndexError,
# because both inherit from it
```

Catching a parent catches every child. That is useful when you genuinely mean “any lookup failure” — and dangerous when you write `except Exception` and accidentally swallow bugs you never considered.

**This is Day 8, applied.** An exception is an object built from a class, and `except ValueError` is really an `isinstance` check. Everything you learned about inheritance — that a child IS its parent — is exactly what decides which `except` block catches what.



# try / except

Catching the right thing · 25 minutes

- try / except
- Specific exceptions
- else and finally
- What not to catch

# try / except — and catching the RIGHT thing

## The basic shape

```
try:
    marks = int(raw)
except ValueError:
    print("That was not a number")
    marks = 0

# the except block only runs if THAT
# exception was raised
```

## Catching several, and seeing the message

```
except (ValueError, TypeError):
    ...           # one block, two types

except ValueError as e:
    print(f"Bad value: {e}")

except KeyError: ... # separate blocks
except IndexError: ... # for different handling
```

## ⚠ Order matters — specific first, general last

```
try: ...
except Exception: # catches EVERYTHING first
    ...
except ValueError: # unreachable - never runs
    ...
```

Python tries each except block in order and takes the first one that matches. Because `ValueError` inherits from `Exception`, a general block written first swallows everything below it. Put the specific types first and the general one last, exactly as you would order an `elif` chain from Day 2.

# The full form: try / except / else / finally

```
try:
    f = open("data.csv")
except FileNotFoundError:
    print("no file")           # only if it failed
else:
    print("opened fine")      # only if it did NOT
finally:
    print("done")            # ALWAYS runs
```

Keep the try block as SHORT as possible — only the line that might fail. Anything else in there can raise an exception you did not mean to catch.

## Which block runs when

|                | No error   | Error caught       |
|----------------|------------|--------------------|
| <b>try</b>     | runs fully | stops at the error |
| <b>except</b>  | skipped    | runs               |
| <b>else</b>    | runs       | skipped            |
| <b>finally</b> | runs       | runs               |

finally runs even if the try block executes a return, or raises something nothing caught. That is what makes it the right place for cleanup.

**You already use finally every day.** Day 9's `with open(...)` is built on exactly this mechanism — the file closes whether the block succeeds, raises, or returns early. When a with statement exists for something, prefer it; write try/finally yourself only when one does not.

# The two ways error handling goes wrong

## ! 1. The bare except

```
try:
    process(data)
except:          # catches ABSOLUTELY everything
    print("something went wrong")
```

This catches your typos, your `NameErrors`, and even `Ctrl+C` — so the program cannot be stopped and the real bug is invisible. If you truly need a catch-all, write `except Exception`, which at least leaves `KeyboardInterrupt` alone.

## ! 2. The silent pass

```
try:
    value = int(row["mark"])
except ValueError:
    pass          # the row is silently skipped
```

Six months later someone notices the totals are wrong and nobody knows why. If you deliberately ignore something, say so in a comment and count it: `bad_rows += 1`. Silence is the enemy of debugging.

### Three questions to ask before you write except

Which exception exactly?

→ name it

What should happen instead?

→ a default, a retry, a message

Would I want to know?

→ if yes, log it or re-raise

3

# Raising Your Own

Signalling a problem · 20 minutes

- raise
- Custom exceptions
- Re-raising

# raise — reporting a problem yourself

## Returning an error message does not work

```
def set_age(age):  
    if age < 0:  
        return "age cannot be negative"  
    return age  
  
total = set_age(-5) + 10  
-> TypeError, far away from the real  
    problem, with a confusing message
```

## raise stops it at the source

```
def set_age(age):  
    if age < 0:  
        raise ValueError("age cannot be negative")  
    return age  
  
set_age(-5)  
-> ValueError: age cannot be negative  
    pointing at the line that was wrong
```

## Pick the type that already describes the problem

```
ValueError    right type, unacceptable value  
TypeError     wrong type altogether  
KeyError      a missing key  
FileNotFoundError a missing file
```

Reusing a built-in type means anyone reading your code — and any code already catching `ValueError` — understands it immediately. Always include a message that says what was wrong AND what was expected. “Invalid input” helps nobody.

# Your own exception types

## A class that inherits from Exception

```
class InvalidMarkError(Exception):  
    """A mark outside 0-100."""  
  
    def record(mark):  
        if not 0 <= mark <= 100:  
            raise InvalidMarkError(f"got {mark}")  
        return mark
```

*That is the whole definition — a name, a parent, and a docstring. No methods needed.*

## Re-raising — handle part of it, then pass it on

```
try:  
    load(path)  
except FileNotFoundError:  
    log("missing file")  
    raise  # bare raise re-throws the SAME error
```

## Why bother, instead of ValueError?

- Callers can catch YOUR error without also catching every unrelated ValueError
- The name documents the rule that was broken
- You can group them: give your project one BaseError and inherit the rest from it

A bare `raise` inside an `except` block re-throws what you just caught, keeping the original traceback intact. Use it when you want to record that something happened but still let the caller decide what to do about it — logging is not handling.

# 4

## Modules & Packages

Using code other people wrote · 25 minutes

- import forms
- The standard library
- Your own modules
- pip

# Four ways to import — and which to prefer

| You write                          | You then use               | When                                   |
|------------------------------------|----------------------------|----------------------------------------|
| <code>import math</code>           | <code>math.sqrt(9)</code>  | the default — the source stays visible |
| <code>import numpy as np</code>    | <code>np.array(...)</code> | long names, or a community convention  |
| <code>from math import sqrt</code> | <code>sqrt(9)</code>       | one or two names you use constantly    |
| <code>from math import *</code>    | <code>sart(9)</code>       | never — see below                      |

## ⚠️ Why `import *` is banned everywhere

```
from math import *
from numpy import *

sqrt(-1)      # whose sqrt is this now?
```

Every name lands in your namespace at once, silently shadowing anything with the same name — including your own variables. You lose all trace of where a function came from.

## The conventions you will see everywhere

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

*Use these exact aliases. Every tutorial, answer and colleague assumes them.*

# The standard library — already installed, already tested

## math

`sqrt, pi, floor, isclose`

*Day 1's float comparison*

## random

`randint, choice, shuffle, sample`

*Day 2's guessing game*

## datetime

`dates, times, differences`

*timestamps in a log*

## csv

`reader, writer, DictReader`

*Day 9*

## json

`loads, dumps`

*web APIs and config files*

## collections

`Counter, defaultdict`

*Days 5 and 6*

## statistics

`mean, median, stdev`

*a preview of Module 2*

## os / pathlib

`paths, folders, existence`

*Day 9*

*Before you write a helper function, spend two minutes checking whether the standard library already has it. It will be better tested than yours, and every Python programmer will recognise it.*

# Your own modules and packages

## A module is just a .py file

```
# helpers.py
def clean(name):
    return name.strip().title()
```

```
# main.py - same folder
import helpers
helpers.clean(" ravi ")
```

*The filename becomes the module name. No registration, no configuration.*

```
if __name__ == "__main__":
```

Code under this line runs when the file is executed directly, but NOT when it is imported. It is how a file can be both a reusable module and a runnable script.

## A package is a folder of modules

```
myproject/
  __init__.py      marks it as a package
  loading.py
  cleaning.py

from myproject import cleaning
```

`__init__.py` can be completely empty — its presence is what matters. This is how pandas and numpy are organised internally.

## Installing what is not built in

```
pip install pandas      # terminal
!pip install pandas     # inside a Colab cell
# Colab already has numpy, pandas, matplotlib
```

# Debug corner — eight exception and import bugs

| You wrote                           | What happens                       | Fix                                           |
|-------------------------------------|------------------------------------|-----------------------------------------------|
| <code>except:</code>                | catches typos and Ctrl+C too       | <code>except Exception:</code>                |
| <code>except: pass</code>           | the real bug is invisible          | log it, or count it                           |
| <code>except Exception first</code> | later blocks never run             | specific types first                          |
| <code>return "error message"</code> | fails later, far from the cause    | <code>raise ValueError(...)</code>            |
| <code>raise ValueError</code>       | works, but no message at all       | <code>raise ValueError("what and why")</code> |
| <code>class MyError:</code>         | not catchable as an exception      | <code>class MyError(Exception):</code>        |
| <code>from math import *</code>     | silently shadows your own names    | <code>import math</code>                      |
| <code>import Helpers</code>         | ModuleNotFoundError — case matters | match the filename exactly                    |

Rows 1 to 3 are the same mistake: catching more than you meant to. All eight are in today's practice notebook.

# Putting it together — a CSV loader that survives bad data

```
import csv

class InvalidMarkError(Exception):
    """A mark that is not a number between 0 and 100."""

def parse_mark(row):
    try:
        mark = int(row)          # may raise ValueError
    except ValueError:
        raise InvalidMarkError(f"not a number: {row!r}")
    if not 0 <= mark <= 100:
        raise InvalidMarkError(f"out of range: {mark}")
    return mark

def load(path):
    """Load rows, skipping bad ones but REPORTING them."""
    good, bad = [], []
    try:
        with open(path, newline="", encoding="utf-8") as f:
            for n, row in enumerate(csv.DictReader(f), start=2):
                try:
                    row["mark"] = parse_mark(row["mark"])
                    good.append(row)
                except InvalidMarkError as e:
                    bad.append(f"line {n}: {e}")    # counted, not hidden
    except FileNotFoundError:
        raise          # caller decides what to do
    return good, bad
```



## Custom exception

names the rule that was broken



## Narrow try

only the line that can fail



## Translate

ValueError becomes a domain error



## Per-row catch

one bad row does not stop the load



## Never silent

bad rows are collected and returned



## enumerate(start=2)

line numbers a human can use



## Bare raise

missing file is not ours to handle

A bad row is expected, so it is handled. A missing file is not, so it goes up to the caller. That distinction is the whole session.

# Practice session — 30 minutes



## Predict the output

Which block runs, and in what order

8 questions



## Spot & fix the bug

Bare except, wrong order, silent pass

4 questions



## Write the code

Safe division, validation, retry, custom error

5 tasks

## Your notebook

[Day11\\_Practice\\_Questions.ipynb](#)

- Name the exception first
- Then decide what to do
- Raise a hand — I am walking around
- Solutions shared after class

***Never write `except:` or `except Exception:` in today's practice. If you cannot name the exception you expect, you do not yet know what you are handling.***

## TODAY IN TWELVE LINES

# Recap

- 1 Read a traceback from the BOTTOM — that line names the error.
- 2 Exceptions are classes, so catching a parent catches its children.
- 3 Catch the SPECIFIC exception you expected, never everything.
- 4 Specific except blocks first, general ones last — like elif.
- 5 else runs only if nothing failed; finally runs always.
- 6 Never write a bare except: — it catches typos and Ctrl+C.
- 7 Never except: pass — count it or log it, but do not hide it.
- 8 raise ValueError("what and why") beats returning a message.
- 9 A custom exception is a class inheriting from Exception.
- 10 A bare raise re-throws what you caught, traceback intact.
- 11 import math is the default; from x import \* is never right.
- 12 A module is a .py file; a package is a folder with \_\_init\_\_.py.

### Homework

Add proper exception handling to your Day 9 CSV functions · Write a custom exception for one rule in your own code · Split a notebook into a helpers.py module and import it

### Next class — Topic 1.11

NumPy: array creation and indexing — np.array, arange, zeros and ones, slicing and reshaping